

INGEGNERIA DELLA CONOSCENZA

Alessandro Carella

INDICE

1. Sistemi intelligenti basati su conoscenza	3
2. Ricerca di soluzioni in spazi di stati	3
2.1. Risoluzione di problemi mediante ricerca	3
2.2. Spazi di stati	3
2.3. Ricerca su Grafo	4
2.4. Algoritmo di ricerca generico	5
2.5. Strategie di ricerca non informate	6
2.5.1. Ricerca in ampiezza	7
2.5.2. Ricerca in profondità	7
2.5.3. Approfondimento iterativo	8

SISTEMI INTELLIGENTI BASATI SU CONOSCENZA

RICERCA DI SOLUZIONI IN SPAZI DI STATI

2.1. Risoluzione di problemi mediante ricerca

La risoluzione di diversi problemi spesso consiste nella *ricerca* in uno specifico modello del mondo. Considerando il caso di un sistema che, dato un *obiettivo* da raggiungere, ragiona su un *modello* del mondo fatto di *stati*, in *assenza di incertezza*.

Per raggiungere questo scopo, si adotta una rappresentazione *piatta*, ovvero relativa a un singolo livello in una gerarchia.

L'*astrazione* del problema prevede la ricerca di un *percorso* che vada da un nodo di partenza a uno dei nodi-obiettivo, detto *goal*, in un *grafo orientato*.

Una strategia di ricerca comune a molti problemi in AI è la seguente: il sistema computa su una *rappresentazione interna*. Il sistema riceve solo una descrizione di *cosa* rappresenti una soluzione; *come* ottenerla sarà il compito di un algoritmo di ricerca. Spesso questi problemi e i relativi algoritmi risolutivi sono NP-completi.

Tali problemi risultano *difficili* anche per agenti umani. Spesso non esistono soluzioni ottimali per cui ci si accontenta di soluzioni *soddisfacenti* in mancanza d'una *struttura* mappabile su caratteristiche del mondo fisico.

2.2. Spazi di stati

Possiamo formulare la nozione di azione intelligente in termini di uno **spazio di stati**. Uno *stato* contiene tutte le informazioni necessarie per predire gli effetti di un'azione e per determinare se soddisfi l'obiettivo. Una ricerca su uno spazio di stati assume le seguenti nozioni:

- Gli agenti hanno una conoscenza perfetta dello spazio di stati.
- In qualsiasi momento, conosce in quale stato si trova; il mondo è quindi pienamente osservabile.
- L'agente ha un insieme di azioni con effetti deterministici conosciuti.

- L'agente ha un obiettivo da raggiungere e può determinare se uno stato soddisfa l'obiettivo.

Una **soluzione** di un problema di ricerca è una sequenza di azioni che porterà l'agente dallo stato corrente allo stato obiettivo.

Un **problema di ricerca su uno spazio di stati** comprende:

- un insieme di stati
- uno stato distinto detto *stato di partenza*
- per ogni stato, un insieme di azioni disponibili all'agente che si trova in tale stato
- una **funzione di azione** che, dato uno stato corrente e un'azione (s, a) , ritorna un nuovo stato in cui si transita per effetto dell'azione
- un **goal** che è una funzione booleana, $goal(s)$, che ritornerà *true* se s è uno stato che soddisfa l'obiettivo e, in tal caso, s è lo **stato obiettivo**
- un criterio che specifica la qualità di una soluzione accettabile; una **soluzione ottimale** massimizza tale criterio, ad esempio se ammette *qualsiasi* sequenza di azioni che porti a uno stato-obiettivo; ad esempio associando *costi* alle diverse azioni da cui discende un criterio del *minimo costo totale* delle azioni; a volte, sono soddisfacenti anche soluzioni sub-ottimali, ad esempio quelle con un costo aggiuntivo del +10% rispetto a quello di una soluzione ottimale.

Inserire esempi del libro/dispense

Possibili *estensioni* dei problemi di ricerca riguardano:

- la possibilità di sfruttare una *struttura interna* agli stati: ad esempio nei video-game;
- il caso di stati *non* completamente *osservabili*: ad esempio il robot-consegne non conosce la posizione iniziale delle consegne, oppure un sistema di tutoring può non conoscere bene le attitudini di uno studente;
- l'aggiunta di azioni *stocastiche*: ad esempio possibilità di commettere errori oppure mancato apprendimento da parte dello studente di un argomento;
- la possibilità di definire, invece degli stati finali, *preferenze aggiuntive* complesse, in termini di *ricompense* o *punizioni*.

Inserire esempi del libro/dispense

2.3. Ricerca su Grafo

Per risolvere il problema, si definisce lo spazio di ricerca e si applica un algoritmo di ricerca. Come detto precedentemente, molti compiti possono essere ricondotti alla *ricerca* di percorsi in un grafo. Un modello *astratto* della soluzione sarà *indipendente* dal particolare dominio applicativo di interesse. Dato un *grafo orientato* fatto di nodi connessi da archi, si dovrà trovare un *percorso* da un nodo di partenza a uno obiettivo. Ci possono essere più modi per rappresentare uno stesso problema.

Un **grafo orientato esplicito** consiste di:

- un insieme di *nodi* N , anche non finito;
- un insieme di *archi* A , ossia di coppie *ordinate* di nodi, anche non infinito;
 - l'arco $\langle n_1, n_2 \rangle$ si dirà **uscente** da n_1 ed **entrante** in n_2
 - si dirà che il nodo n_2 è un **vicino** del nodo n_1 se e solo se $\exists \langle n_1, n_2 \rangle \in A$
- un **percorso** dal nodo s al nodo g , denotato da $\langle n_0, n_1, \dots, n_k \rangle$, è una *sequenza* di nodi tale che se $s = n_0, g = n_k$ e $\forall i = 1, \dots, k : \langle n_{i-1}, n_i \rangle \in A$; in alternativa, si può indicare una *sequenza di archi* $\langle n_0, n_1 \rangle, \langle n_1, n_2 \rangle, \dots, \langle n_{k-1}, n_k \rangle$ o anche *sequenza di etichette* su tali archi: $\langle n_0, \dots, n_i \rangle$ *parte iniziale* di $\langle n_0, n_1, \dots, n_k \rangle$, con $i \leq k$.

Nel grafo esiste un insieme di *nodi-obiettivo* che si possono identificare anche tramite il predicato $goal(\cdot)$, funzione booleana definita sui nodi. Una **soluzione** è un percorso dal nodo di partenza a uno obiettivo.

In alcuni casi è associato un **costo** a ogni arco: dato $\langle n_i, n_j \rangle$

$$cost(\langle n_i, n_j \rangle) \in [0, +\infty[$$

estendibile anche ai percorsi: dato $p = \langle n_0, n_1, \dots, n_k \rangle$

$$cost(p) = \sum_{i=1}^k cost(\langle n_{i-1}, n_i \rangle)$$

Una soluzione **ottimale** p avrà costo minimo: $\neg \exists p' \text{ soluzione con } cost(p') < cost(p)$.

Inserire esempio libro/slide. Inserire esercizio slide

Un **ciclo** è un percorso non vuoto in cui primo e ultimo nodo coincidono: $\langle n_0, n_1, \dots, n_k \rangle$ tale che $k > 0$ e $n_0 = n_k$.

Un **grafo aciclico orientato** (*directed acyclic graph*, DAG) è un grafo orientato senza cicli. Un **albero** è un DAG con un solo nodo, la **radice**, senza archi entranti. I nodi senza archi uscenti sono le sue **foglie**.

2.4. Algoritmo di ricerca generico

Un algoritmo di ricerca generico è un algoritmo indipendente dalla strategia di ricerca dal grafo. Dato un grafo, si esplorano *incrementalmente* percorsi dai nodi di partenza verso nodi-obiettivo.

Si usa una struttura dati per rappresentare la **frontiera**, detta *fringe*, dei percorsi già esplorati. Inizialmente la frontiera contiene percorsi costituiti dai soli *nodi di partenza*. Successivamente si effettua l'*espansione* dei percorsi nella frontiera verso nodi inesplorati, fino a incontrare un nodo-obiettivo:

- si seleziona un percorso (rimuovendolo dalla frontiera);
- si estende il percorso con ogni arco uscente dall'ultimo nodo;
- si aggiungono alla frontiera i percorsi ottenuti.

procedure Search($G, s, goal$)

Input

G : grafo con insiemi di nodi N e di archi A

s : nodo di partenza

$goal$: funzione booleana sui nodi

Output

percorso da s a un nodo per il quale $goal$ sia vera oppure quando non ci sono percorsi

Local

frontiera: insiemi di percorsi

frontiera $\leftarrow \{s\}$

while frontiera $\neq$ (simbolo insieme vuoto) do

 selezionare (e rimuovere) $\langle n_0, \dots, n_k \rangle$ da frontiera

 if $goal(n_k)$ then

 return $\langle n_0, \dots, n_k \rangle$

 frontiera $\leftarrow$ frontiera unito $\{\langle n_0, \dots, n_k, n \rangle : \langle n_k, n \rangle \in A\}$

return (simbolo perpendicolare)

La selezione *non è deterministica*, ha solo impatto sull'efficienza quindi una data strategia di selezione determina il percorso da scegliere. Il annidato può essere interpretato come *temporaneo*, continuando si trovano strade alternative; $\perp$ indica che non vi sono soluzioni; la condizione $goal(n_k)$ viene testata dopo la selezione della frontiera e non all'aggiunta del nuovo nodo:

1. a volte esiste un *arco* verso un nodo-obiettivo ma di *costo elevato*: non sempre conviene terminare restituendo subito il percorso trovato in quanto potrebbe esserci un percorso di costo inferiore;
2. va tenuto conto del fatto che lo stesso test $goal()$ può essere *costoso*; un percorso che termini con un nodo che non sia un nodo-obiettivo e che non abbia vicini andrebbe rimosso.

2.5. Strategie di ricerca non informate

Il problema specifica il grafo e l'obiettivo mentre la **strategia di ricerca** determina il percorso da selezionare dalla frontiera. Le **strategie non informate** non prendono in considerazione la posizione dell'obiettivo. Se si considera un costo unitario per ogni arco, possibili strategie sono la ricerca *in ampiezza*, quella in *profondità* o l'*approfondimento iterativo*. Se è disponibile una funzione di costo, allora si usano strategie dei *costi minimi*.

2.5.1. Ricerca in ampiezza

Nella **ricerca in ampiezza**, detta *breadth-first search*, *BFS* la frontiera è implementata con una *coda*, ossia una struttura FIFO. Si seleziona il *primo* percorso aggiunto. I percorsi sono generati nell'ordine del numero di archi contenuti. A ogni passo, si seleziona uno dei percorsi più corti.

Inserire esempio slide

La BFS risulta utile quando:

- non si hanno problemi di spazio;
- si cerca una soluzione con numero di archi *minimale*.

La BFS è *poco utile* quando:

- tutte le soluzioni sono associate a percorsi lunghi;
- è disponibile conoscenza euristica;
- il grafo viene generato dinamicamente.

2.5.2. Ricerca in profondità

Nella **ricerca in profondità**, detta *depth-first search*, *DFS*, la frontiera è implementata con una *pila*, struttura LIFO: gli elementi vengono aggiunti uno alla volta e quello selezionato e prelevato sarà l'ultimo aggiunto.

Partendo dalla radice, i nodi sono considerati come ordinati da sinistra a destra: il vicino più a sinistra sarà aggiunto in cima per *ultimo*. L'ordine di espansione non dipende dalla posizione dei nodi-obiettivo.

Con una pila, la ricerca procede in profondità:

- *completamento* di un *singolo percorso* prima di provare alternative;
- questo comporta il **backtracking**: si seleziona una prima alternativa per ogni nodo, *tornando indietro* all'opzione successiva solo dopo aver tentato tutti i completamenti;
- l'*ordine di aggiunta* dei vicini alla frontiera/pila non è specificato: questo impatta sull'efficienza e può essere *statico*, ossia prefissato, ovvero *dinamico*, dipendendo dall'obiettivo;
- per una procedura DFS alternativa si veda la figura (inserire figura slide).

Inserire esempio slide

La DFS è *appropriata* in caso di:

- limitazioni di spazio;
- presenza di molteplici soluzioni, anche se costituite da percorsi lunghi;
- ideale quando *tutti* portano a una soluzione;
- oppure se l'ordine di aggiunta dei vicini può essere variato in modo da trovare una soluzione al *primo tentativo*.

La DFS risulta *inefficiente* quando:

- sono possibili percorsi infiniti in caso di grafo infinito o contenente cicli;
- pur esistendo soluzioni alternative poco profonde, la ricerca si attarda su percorsi più lunghi.

2.5.3. Approfondimento iterativo